

Page Tables for Large Memories

1 Core Problem

Problem

TLBs make address translation faster, but they do not solve the size problem: large virtual address spaces can require huge page tables.

If every virtual page needs one page-table entry, a large virtual address space quickly becomes expensive.

Issue	Meaning
Fast translation	The MMU must translate every memory reference quickly.
Large page tables	A large virtual address space has many virtual pages.
Per-process tables	Each process has its own address space, so page-table storage is repeated per process.

Page Count

$$\text{number of virtual pages} = \frac{\text{virtual address space size}}{\text{page size}}$$

2 Two Main Solutions

Method	Main idea	Best when
Multilevel page table	Split the page table into levels and allocate lower-level tables only for used regions.	The virtual address space is sparse.
Inverted page table	Keep one entry per physical page frame instead of one entry per virtual page.	Physical memory is much smaller than the virtual address space.

3 Multilevel Page Tables

Definition

A **multilevel page table** breaks one large page table into a hierarchy. The top level points to lower-level page tables only where the process actually uses memory.

The goal is simple: do not keep page tables for huge unused regions of the virtual address space.

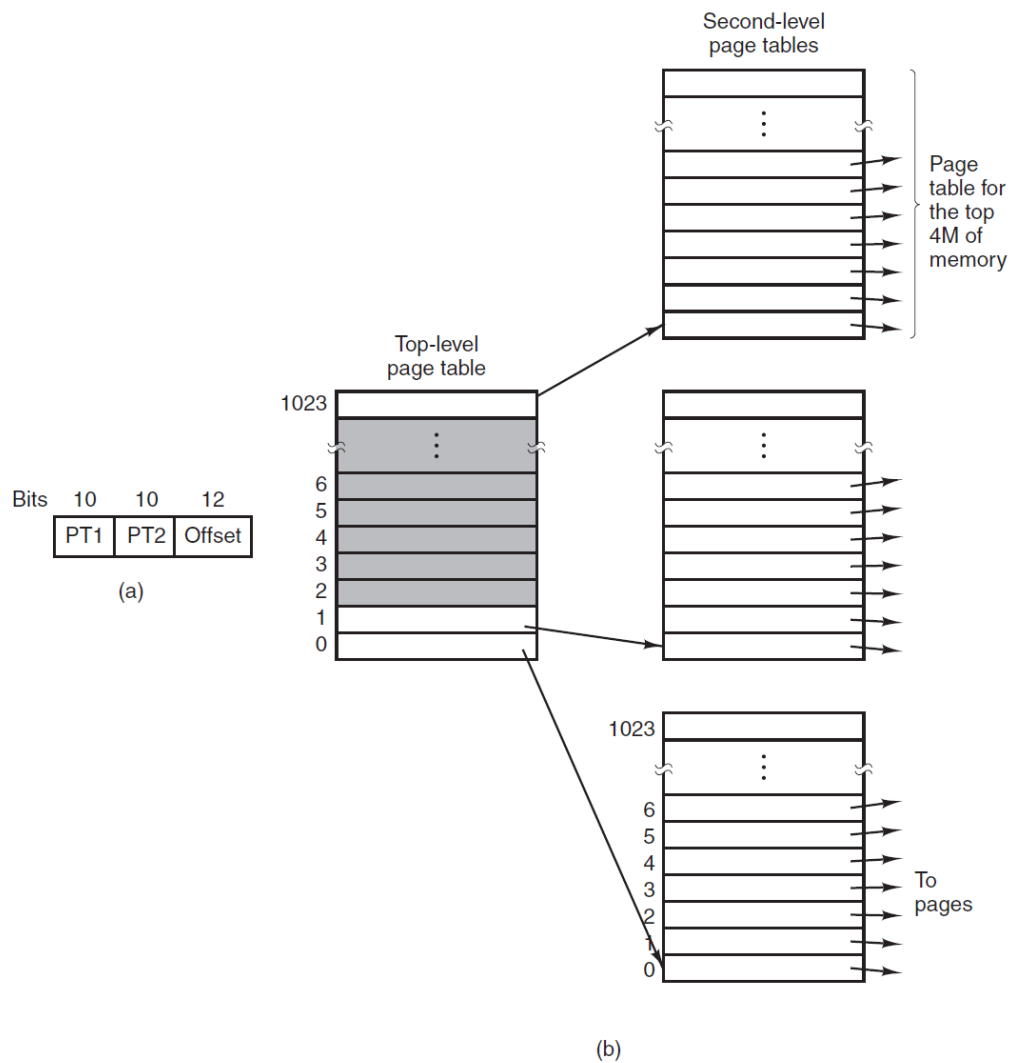


Figure 3-13. (a) A 32-bit address with two page table fields. (b) Two-level page tables.

4 Address Split

In the example, a 32-bit virtual address is split into three fields:

Field	Bits	Meaning
PT1	10 bits	Index into the top-level page table.
PT2	10 bits	Index into the selected second-level page table.
Offset	12 bits	Byte position inside the selected 4-KB page.

Why Pages Are 4 KB

The offset field has 12 bits, so one page contains $2^{12} = 4096$ bytes = 4 KB.

Address Capacity

$$2^{10} \times 2^{10} \times 2^{12} = 2^{32} \text{ addressable bytes}$$

5 How Two-Level Translation Works

1. The MMU extracts the **PT1** field.
2. PT1 indexes the top-level page table.
3. The selected top-level entry points to a second-level page table.
4. The MMU extracts the **PT2** field.
5. PT2 indexes the selected second-level page table.
6. The second-level entry gives the physical page frame number.
7. The page frame number is combined with the unchanged offset.

Final Physical Address

$$\text{physical address} = (\text{page frame number} \times \text{page size}) + \text{offset}$$

6 Why It Saves Space

Suppose a process uses only:

- bottom 4 MB for program text;
- next 4 MB for data;
- top 4 MB for stack.

The middle of the address space is a large unused hole.

Space Saving

Only the top-level page table and the second-level tables for used regions must exist. Unused 4-MB regions do not need second-level page tables.

In the figure, the system needs:

Table	Purpose
Top-level table	Covers the whole 4-GB virtual address space in 4-MB chunks.
Second-level table for entry 0	Maps the program-text region.
Second-level table for entry 1	Maps the data region.
Second-level table for entry 1023	Maps the stack region near the top of memory.

7 Worked Example: Address 0x00403004

Problem

Translate the virtual address 0x00403004 using the two-level split:

$$\text{PT1} = 10 \text{ bits}, \quad \text{PT2} = 10 \text{ bits}, \quad \text{Offset} = 12 \text{ bits}.$$

Solution

First convert the page size:

$$4 \text{ KB} = 4096 = 2^{12}$$

The offset is the lower 12 bits. In hexadecimal, 12 bits correspond to the last 3 hex digits.

$$0x00403004 \Rightarrow \text{offset} = 0x004 = 4$$

Remove the offset by shifting right 12 bits:

$$0x00403004 \div 0x1000 = 0x403$$

Now split the remaining 20 page-number bits into PT1 and PT2:

$$0x403 = 1 \times 1024 + 3$$

So:

$$\text{PT1} = 1, \quad \text{PT2} = 3, \quad \text{Offset} = 4$$

Step	Meaning
PT1 = 1	Select top-level entry 1, which covers virtual addresses 4 MB to 8 MB minus 1.
PT2 = 3	Select page 3 inside that 4-MB chunk.
Offset = 4	Select byte 4 inside that page.

The start of page 3 within the selected 4-MB chunk is:

$$3 \times 4096 = 12288$$

The absolute virtual address is:

$$4 \text{ MB} + 12288 + 4 = 4,194,304 + 12,288 + 4 = 4,206,596$$

Interpretation

The address 0x00403004 means: use top-level entry 1, then second-level entry 3, then byte offset 4 inside that page.

8 Page Faults in Multilevel Tables

Each top-level entry and second-level entry has validity information.

Case	Result
Top-level entry absent	The whole 4-MB region is not mapped. Access causes a fault.
Second-level entry absent	The selected page is not currently present in RAM. Access causes a page fault.
Invalid address	The process is trying to access memory it is not allowed to use. The OS may signal or kill it.

9 More Than Two Levels

Two levels can be extended to three, four, or more levels. More levels make it possible to represent very large sparse address spaces without allocating page-table memory for unused regions.

System idea	Explanation
Two-level paging	A top-level directory points to page tables.
Three-level paging	Adds another directory layer above the page tables.
Four-level paging	Common in 64-bit style address translation, where the address space is much larger.

Tradeoff

More levels save memory for sparse address spaces, but a TLB miss may require more page-table lookups during the page-table walk.

10 Inverted Page Tables

Definition

An **inverted page table** has one entry per physical page frame, not one entry per virtual page.

Instead of asking, “where is virtual page p ?”, the inverted table records, “which process and virtual page currently occupy this physical frame?”

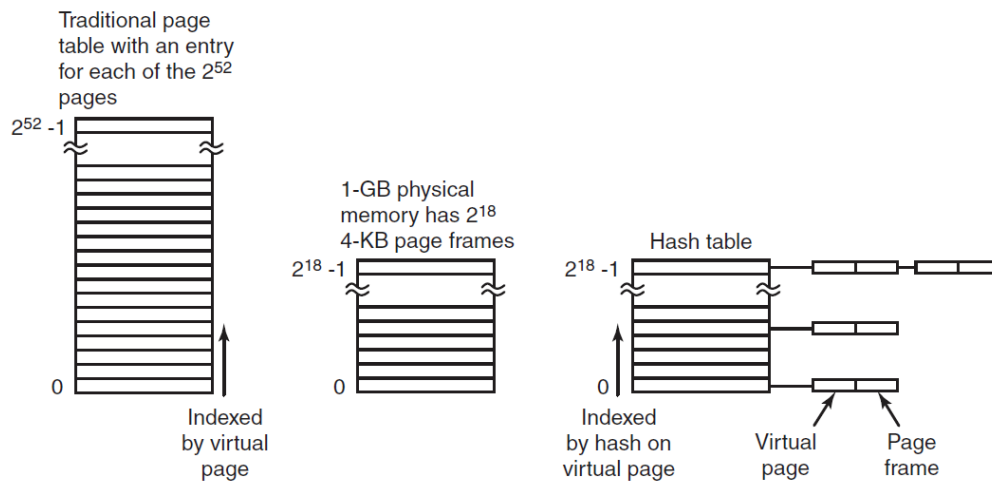


Figure 3-14. Comparison of a traditional page table with an inverted page table.

11 Traditional vs Inverted

Feature	Traditional page table	Inverted page table
Number of entries	One entry per virtual page.	One entry per physical page frame.
Indexed by	Virtual page number.	Usually searched by hashing the process ID and virtual page.
Space cost	Huge for large virtual address spaces.	Depends mainly on physical RAM size.
Lookup difficulty	Direct indexing is easy.	Search is harder without hashing and TLB support.
Typical use	Simple and fast when page-table size is manageable.	Useful for very large virtual address spaces, especially 64-bit systems.

12 Inverted Page Table Example

With:

- 64-bit virtual addresses;
- 4-KB pages;
- 4 GB of RAM.

The number of physical page frames is:

$$4 \text{ GB} = 2^{32} \text{ bytes}$$

$$4 \text{ KB} = 2^{12} \text{ bytes}$$

$$\text{physical frames} = \frac{2^{32}}{2^{12}} = 2^{20} = 1,048,576$$

Meaning

The inverted page table needs about 1,048,576 entries because it has one entry per physical frame, not one entry per possible virtual page.

13 Why Hashing Is Needed

The downside of inverted page tables is lookup.

Lookup Problem

The MMU cannot directly index the table using only the virtual page number, because the table is organized by physical frames.

To find where process n 's virtual page p is located, the system searches for the pair:

$$(n, p)$$

Searching the entire inverted page table on every memory reference would be too slow. A hash table solves this by grouping likely matches into short chains.

Step	Action
1	Hash the process ID and virtual page number.
2	Use the hash value to select a chain of entries.
3	Search only that short chain for the exact (process, virtual page) pair.
4	If found, get the physical page frame number.
5	Insert the translation into the TLB for faster future access.

14 Role of the TLB

Inverted page tables depend heavily on the TLB.

TLB Role

If the requested translation is already in the TLB, the inverted page table is not searched. The expensive search happens mainly on TLB misses.

This is why inverted page tables can be practical: most references hit in the TLB, and only misses need the hash-table lookup.

15 Comparison Summary

Method	Strength	Weakness
Multilevel page table	Saves memory by omitting lower-level tables for unused regions.	Page-table walks may require multiple memory references.
Inverted page table	Table size depends on physical memory, not virtual address space.	Translation is harder and needs TLB plus hashing.

Final Summary

Large virtual address spaces make flat page tables too large.
Multilevel tables save space by allocating only needed lower-level tables.
Inverted tables save space by storing one entry per physical frame.
TLBs are essential because both schemes can make misses expensive.